

Assignment 2

Build and Compare Small Language Models

Student: Zhang Jingxuan

Number: A0326409A

Date: 2025-09-10

Code Repository: [Yo1ogreyZz/DSA4213 Natural-Language-Processing-for-Data-Science](https://github.com/Yo1ogreyZz/DSA4213-Natural-Language-Processing-for-Data-Science)

1. Introduction & Task Overview

This report details the implementation, training, and comparison of two language models: a Long Short-Term Memory (LSTM) network and a lightweight causal Transformer. The objective was to understand model mechanics, evaluate performance via perplexity, and assess text generation.

The models were trained from scratch on the `HuggingFaceTB/cosmopedia` dataset to predict the next token. Their performance was compared quantitatively (perplexity, training time) and qualitatively (generated text samples). This study investigates the impact of architectural choices (LSTM vs. Transformer), tokenization (character vs. word), and regularization (dropout, context length) on model performance. All code is GPU-accelerated and reproducible.

2. Dataset & Preprocessing

2.1. Corpus Loading

- **Corpus:** The experiments used the `stories` configuration of the `HuggingFaceTB/cosmopedia` dataset.
- **Loading Mechanism:** The corpus was loaded using the `datasets` library with `streaming=True` and a character limit of 2,000,000 to manage memory.
- **Reproducibility:** To ensure consistent runs, the fetched text was concatenated and saved to a local file (`data/cosmopedia_corpus.txt`), which served as the source for all experiments.

2.2. Tokenization

Two tokenization strategies were implemented to analyze their effect on model learning:

1. **Character-level (CharTokenizer):** Creates a small, fixed vocabulary from the unique set of all characters in the corpus.
2. **Word-level (WordTokenizer):** Uses a regular expression (`(\w+|[\^\w\s])`) to split text into words and punctuation, building a vocabulary from the most frequent tokens (up to 50,000) and mapping rare words to `<UNK>`.

2.3. Data Splitting and Batching

- **Data Split:** The tokenized sequence was partitioned into training, validation, and test sets (80/10/10 ratio).
- **Sequence Creation:** The `LMSequenceDataset` class converted the token IDs into overlapping input/target pairs (`x` and `y`) for standard autoregressive language model training.
- **Data Loading:** The `torch.utils.data.DataLoader` was configured for CUDA performance, with

`pin_memory=True` and `num_workers=0` for cross-platform stability.

3. Model Architectures & Training

3.1. LSTM

A standard LSTM-based architecture for sequence prediction.

- **Architecture:**

1. `nn.Embedding`: Maps input token indices to dense vectors of `emb_dim`.
2. `nn.LSTM`: A single-layer LSTM processes the sequence of embeddings.
3. `nn.Dropout`: A separate dropout layer regularizes the LSTM's output hidden states.
4. `nn.Linear`: A fully connected layer projects the final hidden state to the vocabulary size, producing logits.

3.2. Transformer

A lightweight, decoder-only Transformer built from scratch.

- **Architecture:**

1. **Token & Positional Embeddings:** Input token embeddings are added to learned positional embeddings to inject sequence order.
2. `nn.TransformerEncoder`: A stack of `nn.TransformerEncoderLayer` modules forms the model's core.
3. **Causal Attention Mask:** A mask is manually created using `torch.triu` to ensure a token can only attend to previous tokens.
4. `nn.LayerNorm`: Applied to the Transformer stack's output for stabilization.
5. `nn.Linear`: A final fully connected layer maps the output to vocabulary-sized logits.

3.3. Training Methodology

- **Device & Performance:** Training automatically uses a `cuda` device if available. Performance on compatible GPUs is enhanced by enabling TF32.
- **Optimization:**
 - **Optimizer: AdamW** with a learning rate of `3e-4`.
 - **Gradient Clipping:** Gradients are clipped to a max norm of `1.0` to prevent exploding gradients.
 - **Mixed Precision (AMP):** `torch.amp.autocast` and `GradScaler` are used on CUDA to reduce memory and speed up computation.
- **Loss Function:** Standard `nn.CrossEntropyLoss` is used to measure the prediction error.

4. Evaluation & Results Overview

Models were evaluated using **Perplexity (PPL)**, calculated as `exp(cross_entropy_loss)`. Lower perplexity indicates a better model. The table below summarizes key results, sorted by **Test PPL**.

Label	Model	Tokenizer	Seq Len	Dropout	Test PPL	Val PPL	Train Time (s)	Batch Size
exp1_lstm_char_L256_d0.0	lstm	char	256	0.0	4.86	4.79	71.22	32
exp2_lstm_char_L256_d0.2	lstm	char	256	0.2	5.01	4.94	72.16	32
exp3_transformer_char_L128_d0.1	transformer	char	128	0.1	5.04	4.97	127.91	32
exp4_transformer_char_L256_d0.1	transformer	char	256	0.1	6.11	6.00	132.32	16
exp5_transformer_word_L128_d0.1	transformer	word	128	0.1	436.72	445.50	69.08	32
exp6_lstm_word_L256_d0.2	lstm	word	256	0.2	619.93	629.61	57.52	32

Overall Best Model: The best-performing model was `exp1_lstm_char_L256_d0.0`, achieving a **Test PPL of 4.86**. This suggests that for this dataset and model scale, a character-level LSTM with a longer context length and no dropout provided the most effective configuration.

5. Training Stability Analysis

The training and validation loss curves for each experiment provide insight into the learning dynamics and stability.

Exp1: The curves for the best model show a smooth, monotonic decrease. The validation and training losses track each other very closely, indicating no significant overfitting, which justifies the use of zero dropout in this case.

Exp2: With dropout enabled, the validation loss is initially lower than the training loss, a common artifact of dropout. The final convergence point is slightly higher than the model without dropout, suggesting that a rate of 0.2 might be too strong for this model size.

Exp3: The Transformer model exhibits stable convergence and its performance is competitive with the LSTM, though it required significantly more training time due to its more complex architecture.

Exp4: Increasing the sequence length for the Transformer resulted in a higher loss. This is likely due to the fixed model capacity (2 layers) being insufficient to handle the longer-range dependencies effectively, or the smaller batch size (16, due to memory constraints) introducing more noise.

Exp5: The word-level models show substantially higher loss values, as the much larger vocabulary makes the prediction task inherently more difficult for a small model.

Exp6: Similar to the word-level Transformer, the word-level LSTM struggles with the large vocabulary, resulting in high perplexity. This highlights the challenge of word-level modeling without sufficient model capacity.

6. Ablation Studies & Quantitative Analysis

1) Dropout (0.0 vs. 0.2)

- Models:** `exp1_lstm_char_L256_d0.0` (PPL: 4.86) vs. `exp2_lstm_char_L256_d0.2` (PPL: 5.01).
- Observation: No dropout (0.0) performed better.** This suggests that for this model size and dataset, the model was not complex enough to overfit significantly, and the regularization from dropout slightly hindered performance.

2) Context Length (128 vs. 256)

- Models:** `exp3_transformer_char_L128_d0.1` (PPL: 5.04) vs. `exp4_transformer_char_L256_d0.1` (PPL: 6.11).
- Observation: The shorter context length (128) performed better** for the Transformer. This is likely because the model's limited capacity could not effectively utilize the longer context, and the required reduction in batch size for the 256-length experiment may have negatively impacted training.

drencess and entific locked it meant recks saids after spark own began and learning and learned them th

- **Top-k (k=50):** Produces some of the most readable text, forming longer phrases.

er worldivation and precialby appries her had worlds of evences leadling of gues cont park. At a knew flunfor felt hisfared in jobative our progressor coatinsoff Excited Sarm their aberstanth (ennia s

- **Top-p (p=0.9):** Similar quality to top-k, generating varied continuations.

]rejing along the remally screeep in a without or her passion other high compartice of discoveration for her suffered experience and hearm her would experiences of sparked everyone meant about it powers

8. Conclusion

This assignment successfully demonstrated the end-to-end process of building, training, and evaluating small language models. The key takeaways are:

1. **Best Performance:** The **LSTM model with character-level tokenization, a sequence length of 256, and no dropout** achieved the best performance with a **Test PPL of 4.86**.
2. **Tokenization is Critical:** For small models and datasets, **character-level tokenization is far more effective** than word-level. It avoids data sparsity and allows the model to achieve much lower perplexity.
3. **Model Complexity vs. Task:** The simple LSTM architecture outperformed the lightweight Transformer in this context, indicating that for smaller-scale tasks, a well-tuned recurrent model can still be highly competitive and more computationally efficient.
4. **Hyperparameter Sensitivity:** Results show that performance is highly sensitive to choices like context length and dropout. The optimal settings depend on the interplay between model architecture, data, and computational budget.
5. **Generation Quality:** Qualitative analysis confirms that while greedy decoding is flawed, temperature and nucleus sampling (top-k, top-p) offer an effective trade-off between coherence and creativity.